ParcelPilot Error Code and Log Message Reference

Error codes, log levels, request correlation, timing, and safe logging.

ParcelPilot operations reference

Contents

1. Using codes

Code	Service	Meaning
API-400	api	Malformed request reaching application logic
API-500	api	Unhandled top-level exception; inspect downstream evidence
API-503	api	Required downstream service unavailable

Using codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

2. Authentication codes

Code	Service	Meaning
AUTH-401	auth	Invalid credentials
AUTH-423	auth	Account temporarily locked
AUTH-503	auth	Authentication service unavailable

Authentication codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Normal operating pattern

Normal authentication traffic includes successful logins, occasional AUTH-401 responses for bad credentials, token refreshes, and a small number of expected account-lock events during testing.

Failure pattern

A service incident is more likely when known-good accounts fail together, AUTH-503 repeats across many request IDs, and the auth health state is degraded or down. A single AUTH-423 is an account-state problem until evidence shows broader impact.

Worked example

If one user is locked at 09:14 but two other known-good users sign in at 09:15, do not classify the event as an authentication outage.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

3. Orders codes

Code	Service	Meaning
ORD-404	orders	Order identifier not found
ORD-409	orders	Database schema migration mismatch
ORD-500	orders	Orders database could not be opened
ORD-503	orders	Orders service unavailable

Orders codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

4. Tracking codes

Code	Service	Meaning
TRK-206	tracking	Partial carrier events
TRK-409	tracking	Carrier snapshot too old
TRK-422	tracking	Carrier snapshot invalid
TRK-504	tracking	Carrier/provider timeout

Tracking codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

5. Document codes

Code	Service	Meaning
DOC-403	documents	Document exists but cannot be read
DOC-404	documents	Requested document not found
DOC-500	documents	Document-store path invalid

Document codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Normal operating pattern

Document routes first confirm the order and then look for the requested file under the configured store. Some orders do not have proof-of-delivery or customs documents, so a controlled not-found response can be normal.

Failure pattern

DOC-404, DOC-403, and DOC-500 should remain separate. A missing file, an unreadable file, and an invalid document root have different causes and different next checks.

Worked example

If label.pdf opens but proof.pdf returns DOC-404, do not change the global document-store path. Verify whether proof.pdf exists for that order.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

6. Notification codes

Code	Service	Meaning
NTF-422	notifications	Notification payload invalid
NTF-429	notifications	Queue backlog over threshold
NTF-503	notifications	Delivery provider unavailable

Notification codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

7. Reporting codes

Code	Service	Meaning
RPT-422	reporting	Invalid report date range
RPT-500	reporting	Analytics query failed
RPT-504	reporting	Analytics query timed out

Reporting codes should be interpreted with the request path, timestamp, and user-visible symptom. The code identifies a broad failure class, not the entire root cause.

Practical interpretation

Check whether the code repeats, whether it appears on the same request as an API error, and whether the corresponding service status or configuration supports the same conclusion.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Normal operating pattern

Reporting reads a separate analytics database. Small date ranges normally complete faster than large ones, and the same report can return different row counts as the requested window changes.

Failure pattern

RPT-422 is a validation problem, RPT-500 is a query failure, and RPT-504 is a timeout after execution begins. A timeout should be investigated with date range, query type, and repeated duration.

Worked example

A 90-day report timing out while 7-day reports complete in 300 ms is not the same as every report failing immediately with RPT-500.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

8. Log levels

INFO records normal operations, WARN records a condition that may affect quality or capacity, and ERROR records a failed operation. Level alone does not tell you user impact.

- A repeated warning can matter if users see the effect.
- One error from a test request may be unrelated to a customer incident.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

9. Request correlation

API and downstream logs share request_id where possible. Correlation helps connect a generic API failure to the more specific service error.

When IDs are missing, use endpoint, order ID, and timestamp carefully. Nearby lines are not automatically the same request.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

10. Timing and frequency

Compare the incident window with normal frequency. A code that appears once a day as expected noise is different from the same code appearing fifty times in ten minutes.

Use durations over several similar requests before declaring a performance problem.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

11. Noise and unrelated errors

Real log streams contain warnings from background jobs, failed test requests, and errors from unrelated services. Filter by route and time before diagnosing.

Do not choose the most dramatic code in the file. Choose evidence that matches the user symptom.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.

12. Safe logging

Logs should not contain passwords, session tokens, API keys, or full secret values. Configuration checks can safely record present, missing, masked, readable, or unreachable.

Incident notes should quote only the useful error message and request identifier, not copy sensitive raw configuration.

Common mistake

Treating one code as proof without checking scope and current evidence can lead to the wrong service being changed.

What to capture

- Timestamp
- Request ID
- Endpoint
- Service
- Code and short message

Operational detail

Error frequency matters. The same code may be harmless background noise at a low rate and incident evidence when it suddenly increases on the affected route. Compare the incident window with a normal window before deciding the code is unusual.

Logs can show cascades. A downstream service error may be followed by an API wrapper error, a retry warning, and a UI request failure. Treat the earliest specific error on the same request path as the stronger clue.

Example review

- Filter by the reported time window.
- Group by service and code.
- Correlate repeated request IDs.
- Compare durations with nearby successful requests.

Keep the original log message available for debugging, but incident output should use a short safe summary rather than dumping raw logs to the user.